KernelScript: Exploring a Typed Unified-Source Programming Model for eBPF

Anonymous Authors

Abstract. Realistic eBPF applications span verifier-constrained kernel programs, userspace loaders, shared maps, skeletons, and sometimes kernel-module code. This paper studies KernelScript, a beta domain-specific language (DSL) that explores whether these pieces can be represented in one type-checked source model and lowered to conventional libbpf-compatible artifacts. The scientific question is whether such a unified model can expose cross-boundary eBPF structure to the compiler without abandoning the ordinary Linux BPF toolchain.

At commit ccb15b4, 43 of 44 repository examples compile from KernelScript source and 41 of 44 build into generated C/eBPF projects. The checked-in evaluation then tests selected early rejections, strict verifier loading, isolated XDP attachment, matched XDP and TC traffic, perf-event counters, ring-buffer event delivery, and tcp-congestion struct_ops compatibility. 37 of 41 build-success generated objects verifier-load under a strict pinned-program criterion, and 27 of 27 verifier-clean single-section XDP objects attach and detach in isolated namespaces. For struct_ops, shared libbpf runners distinguish direct object load/attach/detach, loopback TCP byte-transfer behavior, callback reachability under clean and loss-trigger profiles, and a local skeleton-header repair that builds 2/2 affected generated userspace projects. The results support an artifact/prototype claim: KernelScript centralizes recurring project structure for this repository corpus and exposes concrete compatibility limits, but it does not yet establish broad runtime equivalence or broad struct_ops portability.

1 Introduction

eBPF gives Linux developers a controlled way to execute application-specific logic in the kernel [6]. It is now used for packet processing, traffic control, tracing, profiling, security policy, and scheduler experimentation. The programming model remains difficult because the artifact boundary is split: an eBPF program is written under verifier constraints, a userspace loader must open and attach it, maps provide shared state, BPF Type Format (BTF) describes kernel types, and newer interfaces such as kfuncs and struct_ops require close coordination between generated skeleton code and kernel headers.

The usual C/libbpf workflow is powerful but verbose [4]. The developer writes kernel-side C, userspace C, Makefiles, pinning logic, map setup, attachment code, and cleanup paths. Libraries in Rust, Go, or Python improve parts of the workflow [1, 3], but they usually preserve the split between the eBPF program and the controller. Tracing languages such as bpftrace provide a higher-level experience, but intentionally narrow the target domain. The systems question is therefore not whether eBPF needs another surface syntax. The question is whether a language can encode cross-boundary eBPF application structure in a compiler-checkable way that remains compatible with the existing kernel toolchain.

1.1 Why a Unified Source Model?

A single source file is not automatically better than multiple files. The research value comes from making cross-file invariants visible to the compiler. For example, a map declaration is simultaneously a kernel data structure, a userspace file descriptor lookup, a pinning decision, and a type contract between programs. A program reference is simultaneously a source-level function name, a generated BPF program section, a skeleton field, a file descriptor, and possibly a BPF link. In C/libbpf, these relationships are implicit naming conventions and generated-header dependencies. In KernelScript, they can be first-class language values.

This observation motivates the paper's main scientific question: can a typed unified-source model make cross-boundary eBPF application structure explicit to the compiler while preserving the ordinary Linux BPF toolchain path? A tracing-only language such as bpftrace [2] can be concise by excluding XDP, TC, struct_ops, and kfunc workflows. A host-language binding can be ergonomic by focusing on userspace loading. KernelScript attempts a harder point in the design space: preserve multiple eBPF program types and still give the compiler enough structure to generate the loader, maps, and build logic.

KernelScript is a recent DSL that makes this question concrete. It lets developers write eBPF entry points, helper functions, userspace control flow, maps, tail-call style delegation, ring buffers, perf events, kfunc declarations, and kernel-module code in one source language. The compiler lowers this source to eBPF C, userspace C, optional module C, a Makefile, and then the normal libbpf and bpftool path can produce objects, skeletons, and binaries. Because KernelScript is explicitly beta software, its most useful research role today is as a prototype for studying the design space rather than as a production-ready replacement for libbpf.

This paper makes three contributions. First, it formulates KernelScript's central systems idea as a typed unified-source model for multi-artifact eBPF applications, where programs, maps, loaders, pinning choices, and optional kernel-extension code are represented before artifact boundaries are split. Second, it analyzes how the public compiler uses that model to check selected cross-boundary invariants, including program-handle use, program signatures, map access types, ring-buffer API use, config mutability, helper-scope restrictions, kernel-context allocation checks, perf-event group bounds, and stack-limit violations before load or attach time. Third, it implements a reproducible artifact evaluation of test coverage, ex-

ample buildability, generated-code expansion, example marker coverage, static rejection cases, strict verifier loading, isolated XDP attachment, matched XDP and TC traffic runs, perf-event loader and counter workloads, a ring-buffer event-emission workload, direct and callback-instrumented struct_ops checks, and a compiler-source patch that isolates one map-update lowering mechanism. The scientific contribution is not a new BPF verifier or a broad performance-equivalence result. Instead, it is an executable case study of which selected cross-boundary eBPF invariants this prototype exposes and which compatibility limits still leak through the Linux BPF toolchain. The artifact is designed so that a reviewer can rerun the evaluation without changing the KernelScript source tree: all scripts, logs, derived tables, and paper macros are stored outside the cloned repository.

2 Background and Motivation

An eBPF application normally contains at least two components. The first is a restricted kernel program, compiled for the BPF target and accepted only if the kernel verifier can prove memory safety, bounded execution, and legal helper usage. The second is a userspace coordinator that loads objects, finds maps, sets up BPF links, handles signals, and tears resources down. The two components communicate through maps, ring buffers, perf events, or global data.

This split creates three kinds of complexity. **Lifecycle complexity** arises because operations must occur in a valid order: an object must be opened, loaded, attached, read, and detached with type-appropriate APIs. **Representation complexity** appears when the same concept, such as a map or a typed context field, must be represented in source C, generated skeleton headers, userspace lookup code, and BTF-derived type declarations. **Compatibility complexity** appears because eBPF features evolve quickly: struct_ops, kfuncs, dynptrs, TCX (a newer traffic-control attach path), and scheduler extensions depend on kernel, bpftool, and libbpf versions.

The verifier is essential, but it is intentionally late in the pipeline. A developer may write code, compile it, generate skeletons, link a loader, and only then receive a verifier rejection or an attach-time failure. A DSL can help if it moves some of these checks earlier without hiding the underlying eBPF execution model. For example, a language can know that a function marked `@xdp` expects an `xdp_md` context and returns an XDP action, model maps as typed global variables, and distinguish userspace functions from eBPF functions and kfunc definitions.

3 KernelScript Overview

KernelScript uses attributes to assign functions to execution domains. A function marked `@xdp`, `@tc`, `@probe`, `@tracepoint`, or `@perf_event` becomes an eBPF entry point. A function marked `@helper` is callable from eBPF code. A normal `main` function is compiled into userspace C. A function marked `@kfunc` can trigger kernel-module generation. The source therefore contains the application structure that would otherwise be scattered across multiple files.

Listing 1: Minimal shape of a unified-source eBPF application.

```
include "xdp.kh"

@xdp fn pass_all(ctx: *xdp_md) -> xdp_action {
  return XDP_PASS
}

fn main() -> i32 {
  var prog = load(pass_all)
  attach(prog, "lo", 0)
  detach(prog)
  return 0
}
```

Maps are declared as typed globals rather than as separate C sections and userspace lookup code. A KernelScript program can declare a hash map from `u32` to a struct, use it in an XDP function, and read or update it in userspace. The compiler lowers the declaration to eBPF map definitions, skeleton references, and helper functions. The same pattern applies to ring buffers and perf events: high-level operations remain explicit in source, but the generated code performs the required libbpf calls.

The language also experiments with lifecycle typing. In the source model, `load` returns a program value and `attach` consumes a compatible loaded program and target. The repository contains tests for detach API generation and type errors around program references. The implementation is not a proof of full typestate soundness, but it shows the research direction: lifecycle ordering can be treated as a language-level property rather than only as a runtime convention.

4 Design Principles

The design principles map directly to the three complexities above. Lifecycle complexity motivates explicit domains and lifecycle typing, representation complexity motivates typed maps, program handles, and generated project structure, and compatibility complexity motivates preserving the kernel toolchain while exposing escape hatches for fast-moving kernel interfaces.

Preserve the kernel toolchain. KernelScript does not replace the kernel verifier, clang, bpftool, or libbpf. Instead it generates C and Makefiles that flow through the same toolchain used by C/libbpf projects. This choice gives the prototype immediate access to BTF, skeleton generation, and existing kernel diagnostics. It also means compatibility failures remain visible and measurable.

Make domains explicit. The attribute system separates userspace, eBPF, helper, and kernel-module code. This matters because the same surface syntax cannot mean the same thing in every domain. Userspace can allocate and print freely, eBPF

code must obey stack and helper restrictions, and kfunc code targets a module build. A unified source model is useful only if the compiler keeps these domains separate internally.

Move verifier-relevant checks earlier. The compiler performs safety analysis before code generation. In our run, `safety_demo.ks` is rejected because the compiler estimates 608 bytes of stack use, above the 512-byte eBPF limit. The check does not make the verifier redundant, but it can reduce late failures for covered cases that would otherwise require inspecting verifier logs.

Expose low-level escape hatches. eBPF users often need new helpers, kernel functions, or BTF-specific types before high-level libraries expose them. KernelScript supports includes and extern declarations so that a program can refer to generated header declarations. For a research prototype, this escape hatch matters because the Linux BPF API surface changes faster than a DSL can stabilize.

5 Implementation

The public implementation is an OCaml compiler built with dune. It parses KernelScript source, resolves imports and includes, builds symbol tables, performs multi-program analysis and type checking, runs safety analysis, generates an intermediate representation, and emits C artifacts. For an input `foo.ks`, the generated project normally contains `foo.ebpf.c`, `foo.c`, a `Makefile`, and, when kfuncs are present, `foo.mod.c`. The Makefile uses bpftool to dump `vmlinux.h`, clang to produce a BPF object, bpftool to generate a skeleton header, and gcc to link the userspace loader with libbpf, libelf, and zlib.

The compiler's architecture aligns with the research hypothesis. It has separate modules for maps, userspace code generation, eBPF C generation, kernel-module generation, context-specific code generation, safety checking, BTF parsing, dynptr bridging, tail-call analysis, and struct_ops registry logic. These modules encode eBPF concepts directly rather than treating the language as a thin preprocessor over C. The design therefore provides places where domain-specific checks can be added: map type compatibility, context field typing, stack estimates, function signature validation, and userspace API generation.

6 Evaluation Methodology

We evaluate KernelScript as a compiler and systems artifact. The goal is to test whether the unified-source model centralizes recurring eBPF project structure, preserves a compile/build path to ordinary libbpf artifacts on our host, and rejects selected pre-load errors. We do not claim packet-rate performance equivalence in this paper, because the current traffic evidence is limited to matched XDP and TC pass/count programs on local veth pairs, and generated-loader runtime evidence is a perf-event lifecycle latency check rather than a dispatch-loop throughput benchmark. Instead, we focus on reproducible evidence that repository examples exercise multiple eBPF domains, selected errors are rejected early, generated artifacts are classified by build and verifier-load outcomes against the local toolchain, matched local workloads cover several program classes, struct_ops checks separate object compatibility, loopback TCP workload behavior, and generated-skeleton compatibility, a skeleton repair checks one version-aware generated-build fix, and one observed runtime gap can be traced to a concrete lowering rule.

The evaluation runs on Linux 6.15.11-061511-generic using clang 18, bpftool 7.7, libbpf 1.3.0, OCaml 4.14, and dune 3.14. The host CPU is a Intel(R) Core(TM) Ultra 9 285K with 24 CPUs, CPU0 governor powersave, turbo enabled, and virtualization reported as none. The microbenchmarks and object workloads do not pin CPUs or change the governor, so their nanosecond and event-rate values should be read as local comparisons rather than general performance claims. The evaluated commit is ccb15b4. The artifact contains eighteen measurement scripts under `experiments/`. They cover compiler tests and example builds, static-check cases, strict verifier loading, isolated XDP attach/detach, XDP and TC traffic over fresh veth/netns pairs, perf-event loader and counter workloads, ring-buffer event emission, direct struct_ops load/attach/detach compatibility, loopback TCP and callback-flag workloads for tcp-congestion struct_ops, a version-aware struct_ops skeleton repair, a longer XDP/TC traffic stress rerun, and two map-update lowering ablations. The paper reports the compiler-source patch ablation as the canonical mechanism-isolation result because it changes the compiler path rather than editing generated output. The traffic scripts compare generated objects with matched hand-written C/eBPF baselines. The perf-event counter, ring-buffer, and struct_ops scripts use one libbpf runner for both objects and check BPF map counts, submitted/received event counts, load-/attach/detach outcomes, TCP socket byte-transfer oracles, or callback-reachability flags. The main mechanism-isolation script copies the KernelScript compiler source, applies a tracked compiler-source patch for array-map increment lowering, rebuilds the patched compiler, and reruns the same count benchmark.

We measure twelve outcomes. **Compiler regression coverage** is approximated by the repository test suite. **Early rejection** is measured by static cases whose expected diagnostics are checked by the harness. **Example marker coverage** is approximated by whether examples for XDP, TC, probes, tracepoints, perf events, maps, ring buffers, dynptrs, tail calls, kfuncs, and struct_ops pass through the compiler and generated build. Feature labels are derived from lexical markers in the example source, so they are a coarse corpus classifier rather than a manual feature-completeness audit. **Generated-structure centralization** is measured as an expansion factor: lines of KernelScript source compared to generated C and Makefile lines. **Compatibility** is measured by generated build successes, failures, verifier-load outcomes, and isolated XDP attach/detach outcomes against the local kernel toolchain.

Mechanism isolation is measured by the map-update lowering ablation. **Runtime sanity** is measured by iperf3 over veth/netns for matched XDP and TC pass/count objects, by a perf-event page-fault counter workload, and by a ring-buffer event-emission workload with a submitted/received/drop oracle. The traffic stress rerun repeats the XDP and TC traffic checks with longer per-trial iperf3 runs. **Struct_ops object compatibility** is measured by a direct libbpf runner that loads, attaches, and detaches the generated tcp-congestion-control struct_ops object and a minimal C/eBPF object with the same function set, avoiding generated skeleton code. **Struct_ops TCP workload compatibility** is measured by selecting the registered BPF congestion-control algorithm on a loopback TCP sender socket, transferring a fixed byte count, and checking algorithm selection, byte count, and detach success for generated and C/eBPF objects. **Struct_ops callback reachability** is measured by instrumenting the generated and C/eBPF tcp-congestion callbacks with BPF array-map flags and checking that the clean loopback workload reaches `cong_avoid` and `cwnd_event` while a 5% loss-injected loopback workload reaches `ssthresh`, `cong_avoid`, `set_state`, and `cwnd_event` for both variants. The loss setting is a callback-trigger profile chosen to enter loss-recovery paths, not a robustness or TCP-performance sweep. **Struct_ops skeleton repair** is measured by rebuilding the two generated struct_ops userspace projects, detecting whether the installed libbpf skeleton header exposes map-link fields, and applying a minimal generated-header repair only when that field is absent. **Generated-loader lifecycle** is measured by a perf-event loader latency check that requires successful load/attach, positive page-fault counter reads, clean detach, and end-to-end invocation timing for generated and hand-written loaders.

The harness treats generated data as first-class evidence. It writes raw stdout and stderr logs for every compiler and make invocation, CSV rows for each example, JSON summaries for scripts, and a small TeX file containing the numeric macros used in this paper. This organization is deliberately close to artifact-evaluation practice: the paper does not rely on a spreadsheet that was manually edited after the run, and the reader can trace each aggregate number to a row in `results/examples_summary.csv`. We also keep temporary build products under `results/build` so that failures can be inspected after the harness exits.

Source lines of code (SLOC) are counted as nonblank, non-comment lines. Generated `vmlinux.h` and generated skeleton headers are excluded from the expansion factor because they are bpftool outputs rather than KernelScript compiler outputs. The expansion therefore counts only source and build logic produced directly by KernelScript: userspace C, eBPF C, optional kernel-module C, and Makefile lines. This definition is conservative for the generated-structure claim because it omits the large skeleton header that a low-level build still needs.

7 Results

Result map. The evaluation is organized by four claims. The example build matrix and generated-code expansion test the unified-source structure claim. The unit tests, static rejection corpus, and safety rejection test the early-checking claim. The verifier, attach, perf-event, ring-buffer, and struct_ops checks test local compatibility. The microbenchmarks, compiler patch, traffic runs, and object workloads test runtime behavior and mechanism isolation without claiming broad performance equivalence.

Test suite. The compiler test suite reports 85 successful suites and 1095 passing tests with no reported failures. The tests cover lexer/parser behavior, type checking, maps, map assignment, userspace code generation, context field typing, safety checking, ring buffers, perf event attach, detach APIs, kfunc attributes, struct_ops, BTF parsing, and other regression cases. This breadth is important because the evaluation of a DSL depends on whether the implementation actually encodes domain concepts rather than only accepting example syntax.

Static rejection corpus. The static-check harness compiles 28 targeted programs and verifies that every observed outcome matches the expected result. The corpus includes one positive control and 27 expected compiler rejections: 5 lifecycle API misuses, 5 program-signature violations, 3 map type or symbol errors, 4 general type-system errors, 2 symbol-validation errors, 1 config-boundary violation, 1 helper-scope violation, 2 kernel-context allocation violations, 2 perf-event group-bound violations, 1 ring-buffer API misuse, and 1 stack-limit violation. These cases are not a soundness proof, but they make the early-rejection claim falsifiable: a future compiler change that accepts one of these invalid programs or changes the diagnostic contract will fail the paper-number generation path.

Example buildability. Table 2 summarizes the example build results. Of 44 examples, 43 compile from KernelScript source. 41 then build into generated C/eBPF artifacts. The single compile-time rejection is `safety_demo.ks`, where safety analysis detects 608 bytes of stack usage and halts before C generation. This is a positive result for the early-checking hypothesis: the compiler catches a verifier-relevant problem early and provides a specific repair suggestion.

The two generated build failures are `sched_ext_simple.ks` and `struct_ops_simple.ks`. Both compile to eBPF objects and generate skeletons, but the generated userspace skeleton code references a `struct bpf_map_skeleton.link` field not available in the installed libbpf 1.3.0 headers. The fast-moving struct_ops path therefore exposes a compatibility issue, not a parser or type-checking failure, and it shows that a unified-source language still inherits version skew across bpftool, libbpf, and kernel headers.

Struct_ops object compatibility. To separate that skeleton-header failure from object compatibility, `run_struct_ops_compat.py` compiles

Table 1: Claim scope, actors, and oracles used in the evaluation.

Claim	Artifact under test	Baseline or actor	Oracle	Boundary
Project generation	Repository examples lowered to generated projects	Compiler and make harness	Build rows, feature markers, generated source lines	Repository corpus only
Early checks	Unit tests and targeted invalid programs	Dune and static-check harnesses	Test success and expected diagnostic categories	Selected cases, not soundness
Compatibility	Generated objects and generated perf loader	Local kernel/toolchain	Pinned-program load criterion, attach state, loader attach/read/detach and invocation timing	Local host only
Runtime sanity	Generated XDP, TC, perf-counter, and ring-buffer objects	Matched C/eBPF objects or shared libbpf runner	iperf3 JSON, positive map counts, perf-counter equality, submitted/received/drop counts	Local workloads only
Struct_ops compatibility	Generated tcp-congestion object and generated skeleton projects	Minimal C/eBPF object with same function set; installed libbpf header	Shared libbpf runner load/attach/detach; loopback TCP byte-transfer; clean and loss-injected callback flags; repaired userspace build status	Local object compatibility, socket workload, callback reachability for two clean functions and four loss-injected functions, and skeleton build repair only; no scheduler workload

Table 2: Repository examples at commit ccb15b4: 41/44 build fully, with one intended safety rejection and two local struct_ops/libbpf skeleton mismatches.

Outcome	Count	Fraction
Examples total	44	100%
KernelScript compile success	43	97.7%
Generated project build success	41	93.2%
Safety rejection	1	2.3%
Struct_ops/libbpf mismatch	2	4.5%

`struct_ops_simple.ks` with `make ebpf-only`, compiles a hand-written C/eBPF object with the same minimal tcp-congestion function set, and loads both with one libbpf runner. The runner finds the struct_ops map, calls `bpf_map__attach_struct_ops`, destroys the returned link, and records detach success. On this host, `bpftool` is v7.7.0 but `libbpf-dev` is 1.3.0, and the local `struct bpf_map_skeleton` header reports skeleton map-link support as no. Without generated skeletons, both the generated object and the C/eBPF object load, attach, and detach in 3/3 and 3/3 trials, respectively. This check does not validate scheduler-extension struct_ops, full struct_ops signature modeling, or workload performance, but it narrows the `struct_ops_simple.ks` generated userspace build failure to a skeleton compatibility problem for this host.

Struct_ops skeleton repair. The local `struct bpf_map_skeleton` header reports skeleton map-link support as no. When that field is absent, `run_struct_ops_skeleton_repair.py` removes the generated skeleton map-link assignments that point at struct_ops links and rebuilds the two affected generated userspace projects. On this host, the original generated userspace builds succeed in 0/2 cases. After the repair, 2/2 build. The script removes 2 assignments across the two examples. This is a version-aware local build repair for the observed map-link mismatch, not a scheduler workload or a portability result across libbpf versions.

Struct_ops TCP workload. To move beyond load/attach/detach, `run_struct_ops_workload.py` uses one runner for the generated object and the C/eBPF object. After attaching each tcp-congestion struct_ops map, the runner creates a loopback TCP client/server pair, sets the client socket's `TCP_CONGESTION` option to the newly registered BPF algorithm, verifies that the socket reports the requested algorithm, transfers 1MiB, and detaches the link. Across 10 trials, the generated object selects the BPF algorithm, transfers the full byte count, and detaches in 10/10 trials. The C/eBPF object does the same in 10/10 trials. Median end-to-end invocation times are 38.7ms and 34.0ms, respectively. This is a local socket-level workload oracle, not a claim about production TCP performance.

Struct_ops callback workload. To check that the TCP workload enters BPF callbacks rather than only selecting a registered algorithm, `run_struct_ops_callback_workload.py` instruments the generated and C/eBPF tcp-congestion objects with a five-slot BPF array map. Each callback sets its own flag, and the shared runner clears the map before transfer and reads it before detach. The runner also prints a clean-profile `callback_clean_required` field, while the Python harness computes the profile-specific oracle from the required flag list. Across 10 trials of 4MiB, the generated object completes the byte-transfer oracle in 10/10 trials and sets both `cong_avoid` and `cwnd_event` in 10/10 trials. The C/eBPF object matches those results in 10/10 byte-transfer trials and 10/10 callback-oracle trials. The generated object sets `cong_avoid` and `cwnd_event` in 10 and 10 trials, respectively, matching 10 and 10 C/eBPF trials. The same script then adds 5% loss to the loopback device with `tc netem` as a callback-trigger profile and repeats 5 4MiB trials per variant. Under loss, the generated object completes 5/5 byte-transfer trials and sets `ssthresh`, `cong_avoid`, `set_state`, and `cwnd_event` in 5/5 trials. The C/eBPF object matches with 5/5 byte-transfer trials and 5/5 callback-oracle trials. The `undo_cwnd` flag is not part of the loss oracle, and it appears in 1 generated loss trials and 1 C/eBPF loss trials. This is callback-reachability

Table 3: Example marker coverage in repository examples. A = attempted, KS = KernelScript compiler success, B = generated project build success.

Feature	A	KS	B	Feature	A	KS	B
XDP	38	37	37	TC	5	5	5
Probe	4	4	4	Tracepoint	2	2	2
Perf	2	2	2	Maps	17	16	16
Ringbuf	3	3	3	Dynptr	1	1	1
kfunc	4	4	3	Struct_ops	2	2	0
Tail-call	2	2	2	Userspace	39	39	37

evidence for two local loopback profiles, not coverage of every tcp-congestion callback or a TCP performance result.

Verifier-load matrix. Build success is still weaker than kernel acceptance, so `run_verifier_matrix.py` loads generated objects with `bpftool prog loadall` without attaching them. The script counts a load as successful only when `bpftool` returns success and at least one BPF program is pinned, avoiding false positives from empty program sections. Of 43 objects present after evaluation, 38 load successfully under this stricter criterion. Among the 41 objects from fully built generated projects, 37 load successfully (90.2%) and 4 fail: 1 remaining reference-ownership failure, 1 map creation failure, 1 missing kfunc symbol in kernel BTF, and 1 object for which no program is pinned. Across all generated objects, including those from generated projects that did not fully build, the matrix also records 1 struct_ops argument-type rejection. The current run has 0 other verifier rejections. This result narrows the compatibility claim from "builds" to a measured split between verifier-clean objects and generated objects that still need backend fixes.

Isolated XDP attach matrix. Verifier acceptance is still weaker than attachability. `run_attach_matrix.py` therefore takes the verifier-clean, single-section XDP subset, creates a fresh network namespace and veth pair per object, attaches each object to the namespace-side veth with iproute2, confirms `prog/xdp` in `ip -d link show`, detaches it, and removes the namespace. The matrix attaches and detaches 27 of 27 eligible objects (100.0%), with 0 failures. This broadens the deployment evidence beyond one smoke-test loader, but it is not a packet-behavior or throughput test.

Example marker coverage. The examples exercise many eBPF domains. Table 3 reports attempted examples, examples that pass the KernelScript compiler, and examples that fully build against the local toolchain. These counts should not be read as a complete language coverage proof. They show that the artifact is broader than a tracing-only DSL while making the struct_ops compatibility gap explicit.

The coverage table also highlights a weakness in the current benchmark suite: XDP dominates. That skew is common in early eBPF tools because XDP examples are easy to compile and do not require tracepoint discovery or workload generation. A later performance study should rebalance the corpus so that TC, perf events, ring buffers, kfuncs, and struct_ops each have multiple matched programs with comparable hand-written baselines.

Table 4: XDP microbenchmarks and compiler-patch lowering ablation. The experimental compiler patch removes the count gap in this harness.

Bench.	Impl.	Instr.	ns (range)
pass	KernelScript	2	5 (5–6)
pass	C/eBPF	2	5 (5–6)
count	KernelScript	21	12 (12–13)
count	KernelScript compiler patch	11	9 (8–11)
count	C/eBPF	11	9 (9–10)

Generated structure. For successful examples, the median KernelScript source size is 31 nonblank noncomment lines. The median generated project size, counting userspace C, eBPF C, optional module C, and Makefile lines but excluding generated `vmlinux.h` and skeleton headers, is 472 lines. The median expansion factor is 11.3x. These measurements show that the compiler emits a median 472 generated source and build lines from 31 KernelScript source lines, giving generated-structure evidence rather than a direct developer-effort result.

The metric is intentionally conservative. It does not say that every generated line is exactly a hand-written line saved, and it does not compare against an expert-written minimal C baseline. It does show that KernelScript centralizes recurring eBPF project structure. Examples such as perf events have particularly high expansion factors because attaching and reading counters requires substantial userspace coordination code relative to the kernel program itself.

Runtime microbenchmarks and lowering ablation. Table 4 uses BPF_PROG_TEST_RUN for 100000 repetitions over seven trials and reports median average nanoseconds with min–max ranges. The pass rows come from `run_microbench.py`, and the count rows come from `run_compiler_patch_ablation.py`. We intentionally use this single ablation run for all count rows so the current, patched, and C objects share one timing run. The count harness resets the pinned `counts` map before each trial and requires key 0 to equal 100000 afterward. The minimal pass program has the same median in this harness: 2 instructions and 5 (5–6) for KernelScript versus 5 (5–6) for C. The map counter isolates a lowering choice. The current compiler emits lookup plus update, which produces 21 instructions and 12 (12–13). Applying a compiler-source patch that lowers constant array-map increments to a checked lookup plus `__sync_fetch_and_add` reduces the object by 10 instructions (47.6%) and 3ns median (25.0%), matching the hand-written C/eBPF baseline in this harness.

Traffic-driven XDP baseline. To check that the microbenchmark result does not hide an attach-path collapse, `run_xdp_traffic.py` runs the same pass and count programs over real TCP traffic on a veth pair. Each trial creates a fresh network namespace, attaches the XDP object to the receiver-side veth, starts an iperf3 server in the namespace, and runs a host-side iperf3 client for 1s. The count variants additionally read the `counts` map after each trial and require a positive count value. Over 10 trials, the pass medians are

17.8 (4.7–19.0) Gb/s for KernelScript and 18.1 (16.7–18.8) Gb/s for hand-written C/eBPF. The count medians are 17.4 (15.8–18.4) Gb/s for KernelScript and 17.5 (16.8–19.0) Gb/s for C/eBPF. In this local run, the count medians differ by 0.6% with overlapping ranges. The corresponding XDP count-map invocation rates are 1.51 and 1.52 Mpps. These numbers are local veth/TCP measurements, not NIC-rate claims, and the pass ranges include local run-to-run noise. The point of the run is narrower: it adds a traffic-driven check to the object-level BPF_PROG_TEST_RUN results.

Traffic-driven TC baseline. To add a non-XDP workload, the same traffic harness runs TC ingress pass and count programs attached with `tc qdisc clsact` and a direct-action BPF filter. Over 10 trials, the TC pass medians are 86.7 (78.1–93.9) Gb/s for KernelScript and 87.4 (82.9–93.5) Gb/s for C/eBPF. The TC count medians are 87.0 (81.4–93.4) Gb/s for KernelScript and 90.6 (82.5–95.4) Gb/s for C/eBPF. In this local run, the count medians differ by 4.0% with overlapping ranges. The count-map execution oracle reports TC BPF-invocation rates of 0.25 and 0.26 Mpps, respectively, and all TC trials have zero retransmits in the checked-in run. This result is still local-host evidence, but it directly addresses the XDP-only limitation of the earlier runtime checks.

Longer traffic stress. To check whether the one-second traffic trials hide immediate instability, `run_traffic_stress.py` reruns the same XDP and TC pass/count objects for 3 trials of 5s each. The stress harness reports iperf3 success for all pass/count runs and positive map-count oracles for all count variants. In the count benchmarks, XDP reports 17.8 (17.0–18.3) Gb/s for KernelScript and 18.1 (18.1–18.2) Gb/s for C/eBPF, while TC reports 86.5 (83.8–91.4) Gb/s and 91.1 (89.3–97.3) Gb/s. In this local run, the XDP and TC count medians differ by 2.0% and 5.0%, respectively, with overlapping ranges. The count-map invocation rates remain positive at 1.54 and 1.57 Mpps for XDP, and 0.25 and 0.26 Mpps for TC. This is still a short local stress check, but it reduces the risk that the traffic evidence is only a one-second artifact.

Perf-event loader and counter workloads. To check generated userspace coordination beyond XDP attachment, the perf-event loader script compiles `perf_page_fault.ks`, compiles a matched hand-written C/libbpf loader, and runs each binary with `sudo -n`. Over 20 trials, both loaders attach two perf-event programs, read page-fault and branch-miss counters, and detach cleanly. The generated loader's median end-to-end invocation latency is 11.1 (8.6–50.9)ms, with p90 44.1ms, while the C/libbpf loader reports 15.2 (11.6–43.0)ms and p90 40.3ms. The corresponding median lifecycle invocation rates are 90.3 and 65.9 invocations/s. Because this timing includes process startup, sudo, load, attach, counter read, detach, and teardown, we treat it as generated-loader lifecycle sanity evidence rather than a dispatch-loop throughput claim.

To add a sustained non-XDP object workload, `run_perf_event_counter.py` compiles matched page-fault counter objects and runs both with one libbpf runner. Each trial faults 65536 pages for 4 rounds, then requires the BPF map count to equal the perf counter read. Over 10 trials, both variants report a median 262147 counted events. The generated object reaches 1.13 (1.10–1.14) million events/s, while the C/eBPF object reaches 1.13 (1.13–1.15) million events/s. In this local run, the medians differ by 0.3% with overlapping ranges. These results are lifecycle and page-fault counter evidence, not a broad perf-event throughput study.

Ring-buffer event workload. To exercise an event-delivery path that uses ref-counted ring-buffer records, `run_ringbuf_workload.py` compiles matched XDP objects that reserve a ring-buffer record, write a marker, submit the record, and increment submitted/drop counters. One libbpf runner executes each object through BPF_PROG_TEST_RUN, consumes the ring buffer, and requires submitted events to equal received events with zero drops, bad markers, bad return values, or runner errors. Over 10 trials of 50000 events, both variants report 50000 submitted and 50000 received events with 0 drops. The generated object reaches 2.08 (2.04–2.11) million events/s, while the C/eBPF object reaches 2.14 (2.01–2.20) million events/s. In this local run, the medians differ by 2.8% with overlapping ranges. This is object-level ring-buffer evidence, and it does not measure generated userspace dispatch-loop throughput.

Generated code size. Among the 41 successful repository examples with disassembly, the median generated eBPF object contains 14.5 instructions. Small examples compile to tiny programs, while map-heavy examples generate much larger objects. The metric helps choose where runtime work is needed: examples with many generated map operations are the most likely to diverge from hand-written C.

Execution smoke test. The separate smoke test compiles `smoke_lo.ks`, builds the generated project, and runs the generated loader with sudo. The loader successfully attaches an XDP pass program to the loopback interface and detaches it: `XDP attached to interface: lo` followed by `XDP detached from interface index: 1`. This test is not a benchmark, but it validates that this minimal generated XDP loader can complete a real load/attach/detach lifecycle on `lo` on the test host, complementing the object-level attach matrix.

8 Discussion

The evaluation supports KernelScript's central research direction, but it also clarifies what remains unproven. The strongest supported claim is a generated-structure claim: one source model can generate the multi-file project structure needed by several eBPF example classes. The test suite, expanded static corpus, and safety demo support a narrower checking claim: the current compiler rejects selected lifecycle, program-signature, map-type, ring-buffer-API, config-boundary, helper-scope, kernel-context allocation, perf-event group-bound, symbol-validation, type-system, and stack-limit errors before kernel loading. The runtime evidence is de-

liberately smaller. It shows that a minimal pass program has no measurable BPF_PROG_TEST_RUN overhead in this harness and that the map-update gap is caused by a specific lowering choice, not by the unified source model alone. The compiler-source patch result strengthens that mechanism claim. The XDP attach matrix plus XDP, TC, perf-event, ring-buffer, direct struct_ops, and callback-instrumented struct_ops checks strengthen the local deployment, generated-loader lifecycle, object-workload, object-compatibility, and callback-reachability evidence across five program classes, and the longer traffic rerun adds a small stress check for the XDP and TC cases. The artifact still does not establish broad runtime equivalence with hand-written C/libbpf.

The struct_ops failures are instructive. A DSL can hide coordination code, but it cannot hide all ABI and API drift. When bpftool-generated skeletons and installed libbpf headers disagree, the generated userspace C fails. A direct libbpf runner can load, attach, and detach the generated tcp-congestion struct_ops object on this host, and a loopback TCP socket can select the registered BPF congestion-control algorithm and transfer data. A callback-flag variant shows that this workload reaches `cong_avoid` and `cwnd_event` in both generated and C/eBPF objects, and the loss-injected profile also reaches `ssthresh` and `set_state`. Thus, the measured failure is not simply an invalid object, unusable TCP algorithm, or entirely unreached callback path. The skeleton-repair experiment shows that one local map-link mismatch can be handled with a version-aware generated-header repair, but a production compiler would still need to integrate that decision into generation, dependency checks, or containerized toolchains. From a systems-research perspective, this is part of the contribution: the artifact exposes where a unified model centralizes structure and where the underlying Linux BPF stack still leaks through.

The expansion-factor metric should also be interpreted carefully. Generated C is not automatically better than hand-written C. It can be verbose, unidiomatic, or harder to debug, so the practical benefit depends on diagnostic quality and source-level mapping from generated errors back to KernelScript source. Future work should therefore measure not only lines of code but time to fix injected errors, quality of compiler messages, and whether developers can understand generated verifier failures.

9 Threats to Validity

No expert C baseline. The expansion factor compares KernelScript source against generated artifacts, not against carefully hand-written C/libbpf programs. The comparison is appropriate for measuring how much project structure the compiler creates, but it is not a direct developer-effort study. An expert could write smaller C for some examples, especially tiny XDP pass/drop programs. Conversely, generated code omits comments and may be denser than maintainable production C. The right interpretation is therefore "generated project structure centralized by the compiler," not "exact lines saved."

Repository examples may be biased. The corpus comes from the public KernelScript repository, so it naturally reflects the features the implementation authors wanted to demonstrate. The corpus is useful for an artifact study because it is reproducible and covers many features, but it is not a representative sample of all eBPF applications. A stronger corpus would include programs from Cilium, bcc/libbpf-tools, production XDP filters, scheduler extensions, and observability agents, then reimplement each in KernelScript and C/libbpf.

Static checks are targeted, not complete. The static rejection corpus is a regression test for selected compiler checks, not a formal proof of lifecycle soundness or memory safety. It strengthens the paper by making early-rejection claims executable across 27 invalid programs, but it does not cover all invalid attach orders, helper contracts, context-dependent verifier failures, or semantic equivalence with hand-written C/libbpf.

The compiler patch is narrow and experimental. The patch used in the ablation is a tracked compiler-source patch applied to a copied compiler tree, not an upstream change in the evaluated KernelScript commit. It is intentionally limited to constant increments on integer array maps, where lookup followed by in-place atomic add preserves the count benchmark's behavior. Broader claims would require upstreaming or otherwise integrating the rule and rerunning verifier, instruction-count, and runtime checks across hash, per-CPU, and structured map values.

Local traffic is not broad workload validation. The verifier matrix loads generated eBPF objects and therefore catches failures beyond C compilation. The attach matrix goes further for verifier-clean single-section XDP objects by installing and removing them on isolated veth devices, and the traffic benchmarks send TCP through matched XDP and TC pass/-count programs. The generated perf-event loader test adds one repository-example loader that opens perf events, attaches BPF programs, reads counters, detaches cleanly, and records end-to-end invocation latency against a C/libbpf loader. The perf-event counter and ring-buffer benchmarks add sustained object workloads, but they use a shared libbpf runner and therefore do not measure generated-dispatch-loop throughput. The struct_ops compatibility checks also use shared libbpf runners. The TCP workload validates socket-level algorithm selection and byte transfer through the generated and C/eBPF tcp-congestion objects, and the callback-flag workload validates that a clean loopback transfer reaches `cong_avoid` and `cwnd_event` in both variants. Its loss-injected profile also reaches `ssthresh`, `cong_avoid`, `set_state`, and `cwnd_event`. The skeleton repair validates a local generated-userspace build fix for the observed map-link mismatch. These checks do not measure production TCP performance, cover every tcp-congestion-control callback, cover scheduler-extension struct_ops, prove compatibility across libbpf versions, or measure generated-loader behavior. However, the traffic runs, including the 5s stress rerun, are local-host veth evidence, not NIC-rate performance or long-duration soak tests. A complete deployment study would run each example in a VM with

known kernel configuration, representative workloads, and explicit cleanup checks after detach.

Kernel version skew matters. The struct_ops failures show that results depend on the alignment between kernel headers, bpftool, generated skeleton code, and libbpf headers. The repair experiment covers the local map-link mismatch, but it is not a substitute for broader version testing. The threat is not incidental: eBPF programming is tightly coupled to kernel versions. A production KernelScript compiler should record the target kernel ABI, check libbpf feature support before generation, and fail early with a dependency diagnostic when a generated feature cannot be compiled locally.

10 Related Work

C/libbpf remains the reference workflow for general-purpose eBPF development [4]. It exposes the full API surface and integrates with BTF and Compile Once, Run Everywhere (CO-RE), but it requires developers to manage multiple artifacts manually. Go and Rust libraries such as cilium/ebpf and Aya improve userspace integration and type safety within their host languages, but still separate the eBPF program from the controlling application [1, 3]. bpftrace takes the opposite approach: it offers a concise high-level language, but focuses on tracing rather than arbitrary XDP, TC, struct_ops, or kfunc-oriented applications [2].

KernelScript occupies a different point in the design space. The language is compiled rather than interpreted, targets more domains than tracing-only languages, and is more opinionated than host-language bindings. Its closest research question is whether eBPF application structure can be made explicit enough for a compiler to generate the low-level project while preserving compatibility with the Linux BPF toolchain. The public KernelScript repository [5] is therefore best read as both an implementation and a concrete design artifact for this space.

11 Conclusion

KernelScript is an early but useful prototype for studying typed unified-source eBPF development. The checked-in artifact supports three bounded claims. First, the public compiler can centralize recurring project structure for this repository corpus: 43 of 44 examples compile, 41 build, and the median successful example expands from 31 KernelScript source lines to 472 generated source and build lines. Second, the implementation exposes selected errors before kernel loading: the targeted static corpus includes 27 expected invalid programs, and the evaluation keeps those diagnostics executable. Third, the generated artifacts are not merely syntactic outputs on this host: verifier loading, isolated XDP attachment, matched XDP/TC, perf-event, ring-buffer, and tcp-congestion struct_ops checks exercise local load, attach, event, traffic, and callback paths against explicit oracles.

The contribution is therefore a reproducible prototype study rather than a finished eBPF replacement. It shows how a compiler-checked unified programming model can make selected cross-boundary eBPF structure explicit while still lowering through libbpf and bpftool. The remaining gates are also clear: version-aware skeleton handling needs upstream integration and broader version coverage, scheduler-extension struct_ops and broader callback-level TCP behavior remain incompletely tested, generated-loader dispatch-loop throughput needs stronger workloads, and the experimental map-update compiler patch needs upstream integration plus broader semantic and runtime validation.

References

[1] Aya contributors. Aya: Rust ebpf library. `https://github.com/aya-rs/aya`, 2026.

[2] bpftrace developers. bpftrace: High-level tracing language for linux ebpf. `https://github.com/bpftrace/bpftrace`, 2026.

[3] Cilium contributors. cilium/ebpf: ebpf library for go. `https://github.com/cilium/ebpf`, 2026.

[4] libbpf developers. libbpf: a bpf co-re userspace library. `https://github.com/libbpf/libbpf`, 2026.

[5] Multikernel Technologies. KernelScript: A domain-specific programming language for ebpf-centric development. `https://github.com/multikernel/kernelscript`, 2026. Accessed June 2026.

[6] The Linux Kernel Documentation Project. eBPF documentation. `https://docs.kernel.org/bpf/`, 2026.